

16장 다차원 배열

1. 다차원 배열의 이해와 활용
2. 3차원 배열



다차원 배열

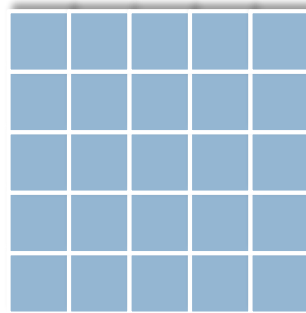
3

- 2차원 이상의 배열 -> 2차원, 3차원, ... , N차원 배열

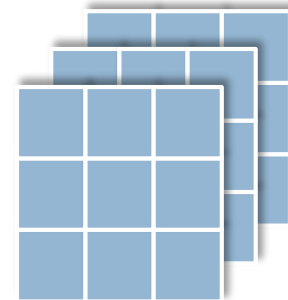
```
int arrOneDim[10];           // 길이가 10인 1차원 int형 배열
int arrTwoDim[5][5];         // 가로,세로의 길이가 각각 5인 2차원 int형 배열
int arrThreeDim[3][3][3];    // 가로,세로,높이의 길이가 각각 3인 3차원 int형 배열
```



1차원 배열 arrOneDim
-> 데이터를 직선으로 나열



2차원 배열 arrTwoDim
-> 1차원 배열을 세로로 나열



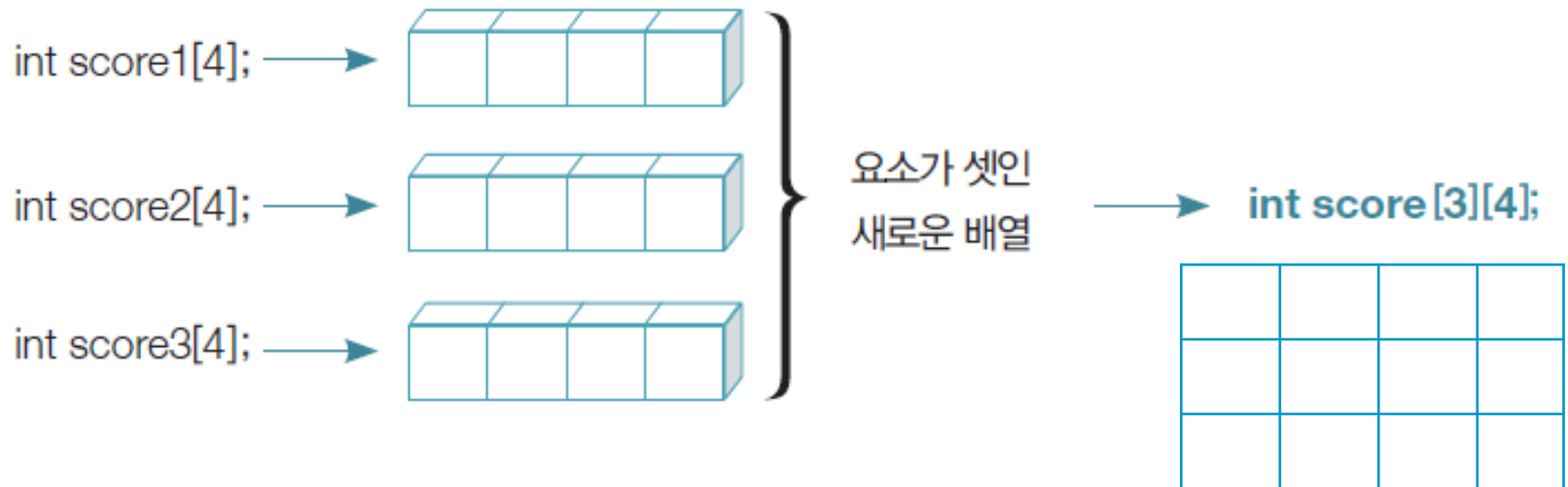
3차원 배열 arrThreeDim
-> 2차원 배열을 높이로 나열



2차원 배열

4

- 2차원 배열은 1차원 배열을 원소(요소)로 갖는 1차원배열
 - ▣ 예) 한 학생의 4과목 점수 저장 -> 크기가 4인 int형 1차원 배열 1개 필요
 - ▣ 예) 학생 수가 3명으로 늘어나면 -> 같은 형태의 1차원 배열이 3개 필요
 - ▣ 예) 학생수가 3000명이면 어떻게 되는가? -> 배열을 2차원 배열로 만들면 더 편리하게 사용





2차원 배열 선언

5

- 2차원 배열 선언 -> 행렬로 이해하는 것이 편리
 - 첫 번째 숫자는 행의 개수(세로길이)
 - 두 번째 숫자는 열의 개수(가로길이)

```
int score [ 3 ] [ 4 ] ;
```

행의 수 (세로길이)

열의 수 (가로길이)





```
#include <stdio.h>
```

 $\{$

```
int arr2[7][9];
```

```
printf("세로7, 가로9: %d \n", sizeof(arr2)); // 배열 전체 크기 계산
```

}

세로7, 가로9: 252

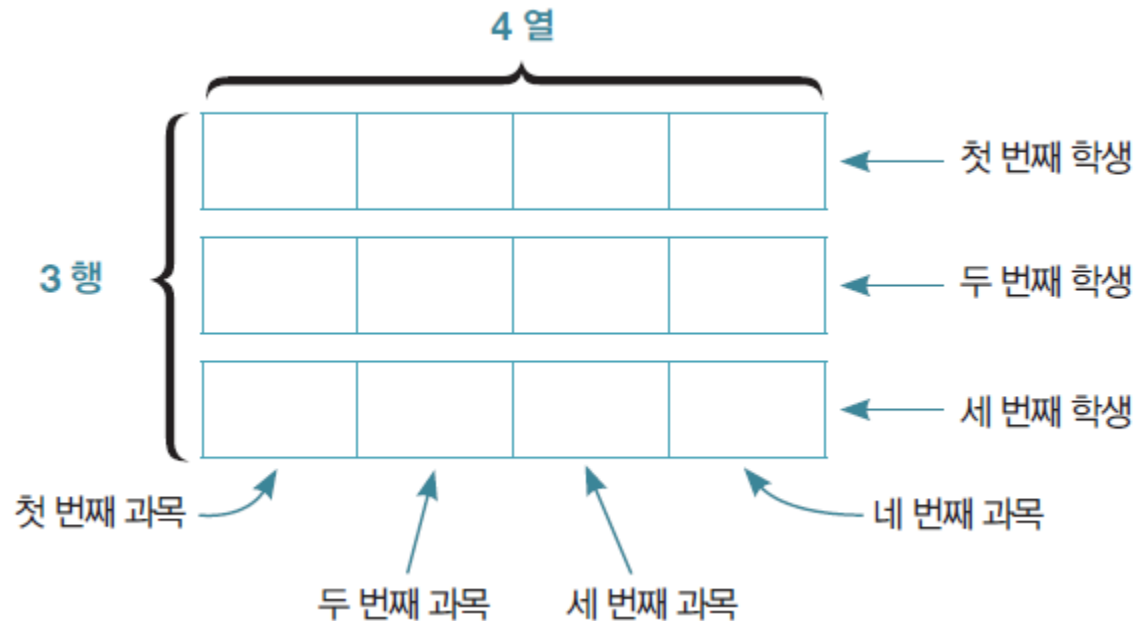
[illegible]



2차원 배열의 응용예

7

- 예) 3명의 학생의 4과목 성적을 저장하는 경우
- 행(넓은 범위)-> 학생, 열(좁은 범위)-> 과목, 원소-> 성적

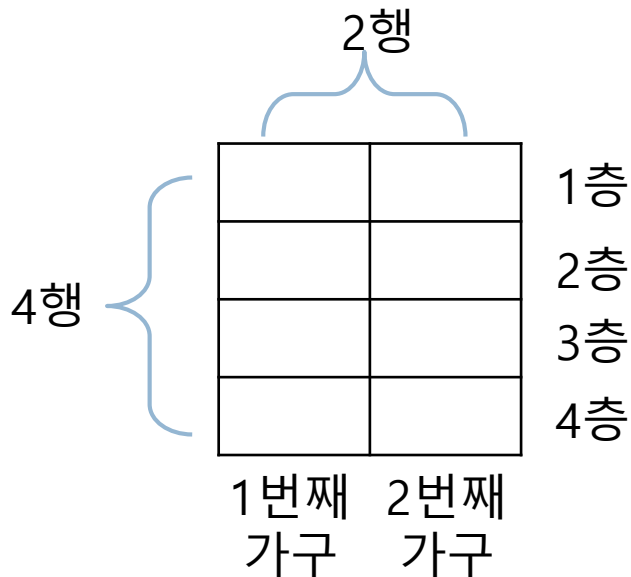




2차원 배열의 응용

8

- 예) 층별로 2가구가 사는 4층짜리 빌라의 거주인원을 저장하는 경우
- 행(넓은범위)->층, 열(좁은범위)-> 가구, 원소->거주인원





2차원 배열요소에 접근

9

- 2차원 배열의 원소를 접근하는 방법
 - ▣ 배열명에 행 첨자(인덱스)와 열 첨자(인덱스) 지정
 - ▣ 위치 지정하는 규칙은 1차원 배열과 같음
 - ▣ 행 첨자의 범위 : 0 ~ (행의 개수 - 1)
 - ▣ 열 첨자의 범위 : 0 ~ (열의 개수 - 1)

	0	1	2	3
0	score[0][0]	score[0][1]	score[0][2]	score[0][3]
1	score[1][0]	score[1][1]	score[1][2]	score[1][3]
2	score[2][0]	score[2][1]	score[2][2]	score[2][3]

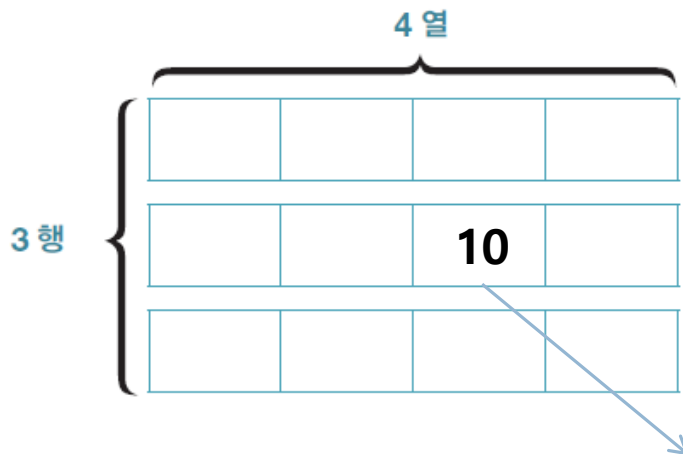


2차원 배열요소에 접근

10

- 2차원 배열의 원소를 참조하는 방법

```
int score[3][4];  
score[1][2] = 10; // 2행 3열의 원소에 10을 대입  
printf("성적: %d\n", score[1][2]); // 2행 3열의 원소 출력
```



두 번째 학생의 세 번째 과목 성적이 10점



2차원 배열요소에 접근

11

- 2차원 배열의 원소에 값을 대입하는 방법
 - ▣ 1차원 배열인 경우 반복문 1개 필요
 - ▣ 2차원 배열인 경우 반복문 2개 필요

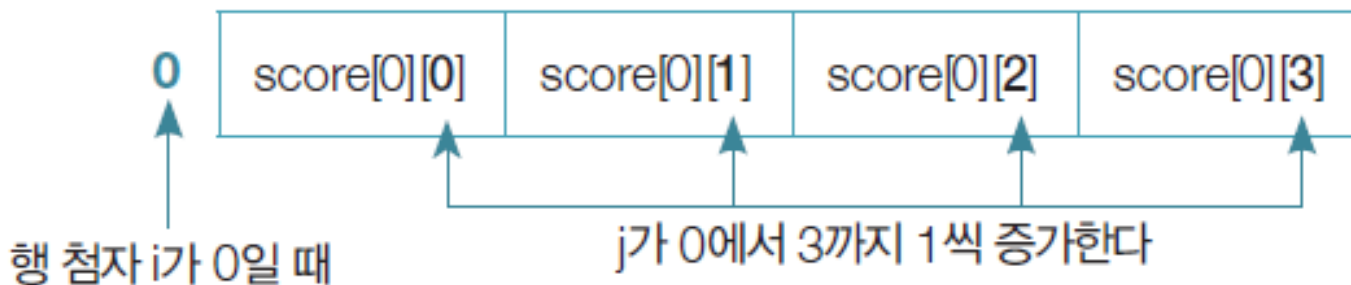


2차원 배열 요소에 접근

12

- 1행의 4개의 원소에 값을 대입하는 코드
 - ▣ 행 첨자를 0으로 고정하고 열 첨자를 0부터 3까지 증가하면서 첫 번째 행의 모든 요소에 입력

```
for (j = 0; j < 4; j++)  
    scanf("%d", &score[0][j]);
```





2차원 배열요소에 접근

13

- 모든 행에 값을 대입하는 코드
 - ▣ 행첨자를 0,1,2로 증가시키면서 모든 행에 점수입력 -> 이중 반복문 필요

```
for(i = 0; i < 3; i++)           // i: 행 인덱스
    for (j = 0; j < 4; j++)       // j: 열 인덱스
        scanf("%d", &score[i][j]);
```

j 반복문 →

	j=0	j=1	j=2	j=3
i=0	score[0][0]	score[0][1]	score[0][2]	score[0][3]
i=1	score[1][0]	score[1][1]	score[1][2]	score[1][3]
i=2	score[2][0]	score[2][1]	score[2][2]	score[2][3]

↓ **i 반복문**



2차원 배열요소에 접근

14

- 2차원 배열의 원소를 출력하는 방법
 - ▣ 이중 반복문을 이용하여 행단위로 줄바꿈하여 화면 출력한다.

```
for(i = 0; i < 3; i++)  
{  
    for (j = 0; j < 4; j++)  
        printf("%5d", score[i][j]);  
    printf("\n");  
}
```

실행 결과	1	2	3	4
	5	6	7	8
	9	10	11	12

예제 2

15

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    int villa[4][2];
```

```
    int popu, i, j;
```

```
    // 가구별 거주 인원 입력 받기
```

```
    for (i = 0; i < 4; i++)
```

```
    {
```

```
        for (j = 0; j < 2; j++)
```

```
        {
```

```
            printf("%d층 %d호 인구수: ", i + 1, j + 1);
```

```
            scanf("%d", &villa[i][j]);
```

```
        }
```

```
    }
```



1층 1호 인구수: 2

1층 2호 인구수: 4

2층 1호 인구수: 3

2층 2호 인구수: 5

3층 1호 인구수: 2

3층 2호 인구수: 6

4층 1호 인구수: 4

4층 2호 인구수: 3

1층 인구수: 6

2층 인구수: 8

3층 인구수: 8

4층 인구수: 7



예제 2

16

```
// 빌라의 층별 인구수 출력하기
```

```
for (i = 0; i < 4; i++)
```

```
{
```

```
    popu = 0;
```

```
    popu += villa[i][0];
```

```
    popu += villa[i][1];
```

```
    printf("%d층 인구수: %d \n", i + 1, popu);
```

```
}
```

```
return 0;
```

```
}
```

1층 1호 인구수: 2

1층 2호 인구수: 4

2층 1호 인구수: 3

2층 2호 인구수: 5

3층 1호 인구수: 2

3층 2호 인구수: 6

4층 1호 인구수: 4

4층 2호 인구수: 3

1층 인구수: 6

2층 인구수: 8

3층 인구수: 8

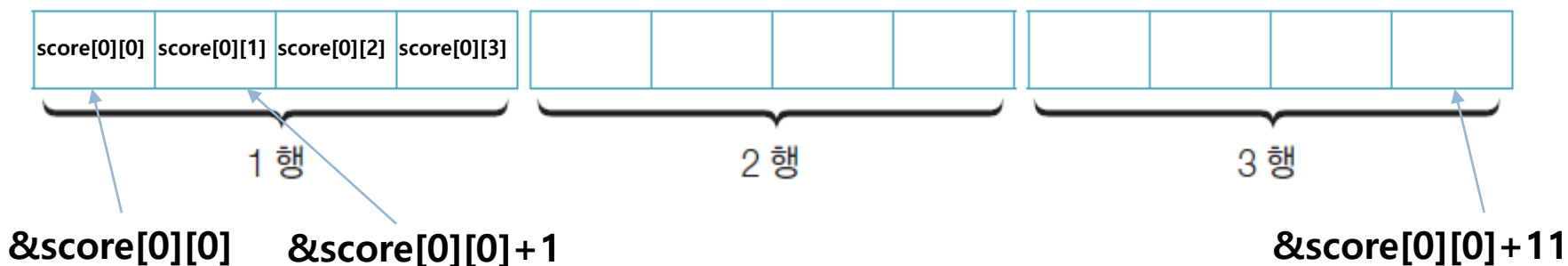
4층 인구수: 7



2차원 배열 메모리상 할당의 형태

17

- 실제 메모리는 1차원 구조이므로 2차원 배열의 각 요소는 아래와 같이 메모리에 행단위로 1차원구조로 저장됨



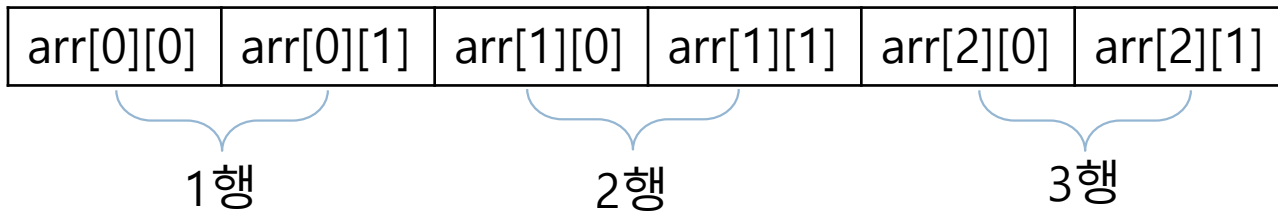
- `int score[3][4];`
 - i행 j열의 주소 -> $\&\text{score}[0][0] + j + 4*i$

 예제 3

18

```
#include <stdio.h>
int main(void)
{
    int arr[3][2];
    int i, j;
    for (i = 0; i < 3; i++)
        for (j = 0; j < 2; j++)
            printf("%p \n", &arr[i][j]);
    return 0;
}
```

```
00F5F8A4
00F5F8A8
00F5F8AC
00F5F8B0
00F5F8B4
00F5F8B8
```





2차원 배열 초기화

19

- 행 구분하는 중괄호를 한 번 더 사용하여 논리적 구조에 맞게 초기화

```
int num1[3][4] = { {1, 2, 3, 4} , {5, 6, 7, 8} , {9, 10, 11, 12} };
```

첫 번째 행

두 번째 행

세 번째 행

1	2	3	4
5	6	7	8
9	10	11	12



2차원 배열 초기화

20

- 행 단위로 초깃값 생략도 가능, 그 행의 남은 요소는 0으로 자동 초기화

```
int num2[3][4] = { {1} , {5, 6} , {9, 10, 11} };
```

1	0	0	0
5	6	0	0
9	10	11	0



2차원 배열 초기화

21

- 행 초기화 괄호 생략된 경우
 - ▣ 기본적으로 1차원 배열을 초기화하는 방식과 같음
 - ▣ 전체 저장 공간의 수만큼 초깃값 나열, 행 단위로 차례로 저장

```
int num4[3][4] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12};
```

1	2	3	4
5	6	7	8
9	10	11	12

→ 첫 번째 행이 먼저 초기화된 후
두 번째, 세 번째 행이 차례로
초기화된다.



2차원 배열 초기화

22

- 저장 공간의 수보다 초깃값이 적은 경우 -> 남은 저장 공간은 모두 0으로 초기화

```
int num5[3][4] = {1, 2, 3, 4, 5, 6};
```

1	2	3	4
5	6	0	0
0	0	0	0

- 배열의 모든 저장 공간을 0으로 초기화하는 것은 간단
 - ▣ 첫 번째 저장 공간만 0으로 초기화하면 됨

```
int nums[100][200] = {0}; → 20000개의 요소가 모두 0으로 초기화
```



2차원 배열 초기화

23

- 행의 수를 생략한 경우
 - ▣ 컴파일러는 행 괄호의 개수로 행의 수를 결정하여 저장 공간 할당
 - ▣ 초기화할지라도 열의 개수는 생략할 수 없음

```
int num3[ ][4] = { {1} , {2, 3} , {4, 5, 6} };
```

행 첨자 생략

행의 개수 세 개

1	0	0	0
2	3	0	0
4	5	6	0

3행 4열의 저장 공간 할당



2차원 배열 초기화

24

- 행의 수가 생략된 채 행 초기화 괄호가 없으면 열의 수에 맞게 초깃값을 꽂아 행의 수 결정
 - ▣ 초깃값의 수에 따라 할당되는 저장 공간 크기가 달라짐

```
int num6[ ][4] = { 1, 2, 3, 4, 5, 6 };
```

첫 번째 행 초깃값

첫 번째 행 초깃값

1	2	3	4
5	6	0	0

} 2행 4열의 저장 공간 할당

 예제 4

25

```
#include <stdio.h>
int main(void)
{
    int i, j;
    // 2차원 배열 초기화의 예 1
    int arr1[3][3] = {
        {1, 2, 3},
        {4, 5, 6},
        {7, 8, 9}
    };
    // 2차원 배열 초기화의 예 2
    int arr2[3][3] = {
        {1},
        {4, 5},
        {7, 8, 9}
    };
    // 2차원 배열 초기화의 예 3
    int arr3[3][3] = { 1, 2, 3, 4, 5, 6, 7 };
```

```
1 2 3
4 5 6
7 8 9
```

```
1 0 0
4 5 0
7 8 9
```

```
1 2 3
4 5 6
7 0 0
```

 예제 4

26

```
for (i = 0; i < 3; i++)
{
    for (j = 0; j < 3; j++) printf("%d ", arr1[i][j]);
    printf("\n");
}
printf("\n");

for (i = 0; i < 3; i++)
{
    for (j = 0; j < 3; j++) printf("%d ", arr2[i][j]);
    printf("\n");
}
printf("\n");

for (i = 0; i < 3; i++)
{
    for (j = 0; j < 3; j++) printf("%d ", arr3[i][j]);
    printf("\n");
}
return 0;
}
```

1	2	3
4	5	6
7	8	9

1	0	0
4	5	0
7	8	9

1	2	3
4	5	6
7	0	0



예제 5: 3명 학생의 4과목 총점과 평균 구하기

27

```
#include <stdio.h>
int main(void)
{
    int score[3][4];           // 3명의 4과목
    int tot;                   // 총점
    double avg;                // 평균
    int i, j;                  // 반복 제어 변수

    for (i = 0; i < 3; i++)    // 학생 수만큼 반복
    {
        printf("네 과목의 점수 입력 : ");
        for (j = 0; j < 4; j++)
            scanf("%d", &score[i][j]);
    }
```

네 과목의 점수 입력 : 72 80 95 60
네 과목의 점수 입력 : 68 98 83 90
네 과목의 점수 입력 : 75 72 84 90
총점: 307, 평균: 76.75
총점: 339, 평균: 84.75
총점: 321, 평균: 80.25



예제 5: 3명 학생의 4과목 총점과 평균 구하기

28

```
for (i = 0; i < 3; i++)  
{  
    tot = 0;  
    for (j = 0; j < 4; j++)  
        tot += score[i][j];  
    avg = tot / 4.0;  
    printf("총점 : %d, 평균 : %.2lf\n", tot, avg);  
}  
return 0;  
}
```

네 과목의 점수 입력 : 72 80 95 60
네 과목의 점수 입력 : 68 98 83 90
네 과목의 점수 입력 : 75 72 84 90
총점: 307, 평균: 76.75
총점: 339, 평균: 84.75
총점: 321, 평균: 80.25



2차원 문자배열

29

- 문자열 1개 저장-> 1차원 문자배열 -> `char str[10];`
- 문자열을 여러 개 저장할 경우 -> 2차원 문자배열 -> `char str[5][10];`
- 4개 문자열 "apple", "blueberry", "orange", "melon"를 저장하는 경우

&str[0][0)	→	a	p	p	l	e	\0				
&str[1][0)	→	b	l	u	e	b	e	r	r	y	\0
&str[2][0)	→	o	r	a	n	g	e	\0			
&str[3][0)	→	m	e	l	o	n	\0				

`char str[4][10];`



예제 6: 4개 문자열 입력하기

30

```
#include <stdio.h>
int main(void)
{
    char str[4][10];
    int i, j;

    for (i = 0; i < 4; i++) {
        printf("%d 번째 문자열 입력: ", i);
        scanf("%s", &str[i][0]);
    }

    for (i = 0; i < 4; i++)
        printf("%d 번째 문자열: %s\n", i, &str[i][0]);

    return 0;
}
```

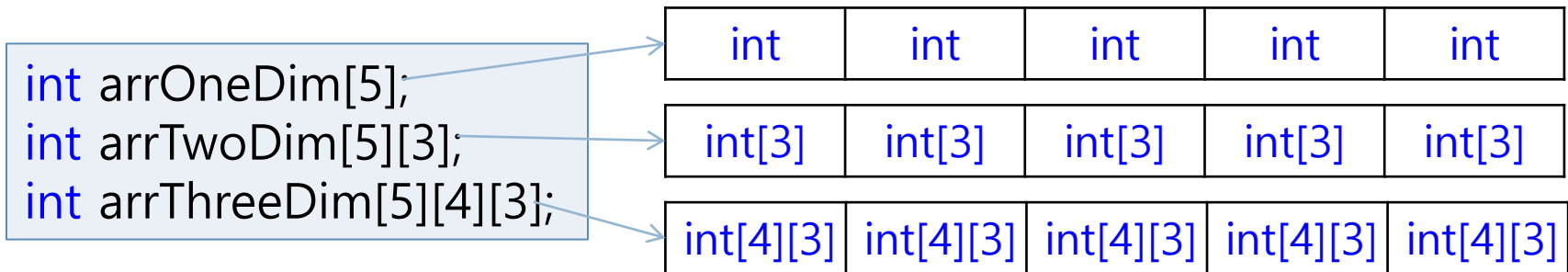
0번째 문자열 입력: apple<엔터>
1번째 문자열 입력: blueberry<엔터>
2번째 문자열 입력: orange<엔터>
3번째 문자열 입력: melon<엔터>
0번째 문자열: apple
1번째 문자열: blueberry
2번째 문자열: orange
3번째 문자열: melon



31

- 각자 공부해볼 것

- N차원 배열은 (N-1)차원 배열을 원소(요소)로 갖는 1차원 배열
 - 2차원 배열 -> 1차원 배열을 원소로 갖는 1차원 배열
 - 3차원 배열 -> 2차원 배열을 원소로 갖는 1차원 배열
 - 4차원 배열 -> 3차원 배열을 원소로 갖는 1차원 배열
 - 모든 다차원 배열은 1차원 배열이고 원소의 자료형이 달라짐





실습과제 1

33

- 다음 행렬 계산을 수행하여 결과를 출력하는 프로그램을 작성하시오. 힌트: 2차원 배열을 3개 선언하고 초기화한 후 이중반복문을 이용하여 연산을 수행한다.
- 배열표현 대신에 포인터 표현을 이용하는 코드도 작성해볼 것

$$\begin{pmatrix} 2 & 4 \\ 5 & -5 \end{pmatrix} + \begin{pmatrix} -2 & 3 \\ 0 & -5 \end{pmatrix} = \begin{pmatrix} 0 & 7 \\ 5 & -10 \end{pmatrix}$$

연산결과:

0	7
5	-10

- 3명 학생의 국어, 영어, 수학 성적을 입력 받아 각 학생의 평균값을 구한 후 최우수 학생의 성적을 출력하는 프로그램을 작성하라.

1번째 학생의 국어, 영어, 수학 성적을 입력: 80 90 100<엔터>
2번째 학생의 국어, 영어, 수학 성적을 입력: 70 80 90<엔터>
3번째 학생의 국어, 영어, 수학 성적을 입력: 60 70 80<엔터>
최우수 학생은 1번째 학생이고 평균점수는 90점이다.



실습과제 3

35

- 다음 행렬을 2차원 배열에 저장하고 원소 중에서 최대값과 위치를 구하는 프로그램을 작성하라.

$$\begin{pmatrix} -5 & 2 & 35 \\ -20 & 5 & 100 \\ -75 & 5 & -25 \end{pmatrix}$$

최대값은 100
위치는 2행 3열

- 예제 6번을 참고하여 4개의 문자열을 입력 받아 문자열의 길이를 구하는 프로그램을 작성하시오.
- 라이브러리 함수를 사용하지 말고 직접 작성할 것
- `char str[4][10];` -> i번째 행의 시작주소 -> `&str[i][0]`

1번째 문자열 입력: apple<엔터>
2번째 문자열 입력: blueberry<엔터>
3번째 문자열 입력: orange<엔터>
4번째 문자열 입력: melon<엔터>
1번째 문자열 길이: 5
2번째 문자열 길이: 9
3번째 문자열 길이: 6
4번째 문자열 길이: 5



실습과제 5

37

- 예제 6번을 참고하여 4개의 문자열을 입력 받아 사전에서 제일 뒤에 나오는 문자열을 구하는 프로그램을 작성하시오.
- 라이브러리 함수를 사용하지 말고 문자열의 첫 문자를 비교하는 방식으로 직접 작성할 것
- `char str[4][10];` -> i번째 행의 시작주소 -> `&str[i][0]`

1번째 문자열 입력: *apple*<엔터>
2번째 문자열 입력: *blueberry*<엔터>
3번째 문자열 입력: *orange*<엔터>
4번째 문자열 입력: *melon*<엔터>
사전에서 제일 뒤에 나오는 문자열: *orange*



과제제출방법

38

- 아래 주소의 작성방법대로 github에 제출할 것
 - ▣ <https://github.com/2sungryul/cprogramming/tree/main>
- 기한 : 이클래스 과제 게시판의 마감시간 참조
- 소스코드의 첫 부분은 아래처럼 제목,날짜,작성자(학번,이름)를 작성할 것

```
// *****  
//   제   목   : 정수 4개의 평균을 구하는 프로그램  
//   날   짜   : 2023년 9월10일  
//   작성자   : 15010101 홍길동  
// *****  
  
// 소스코드 작성
```